

Shooting script

Computer-Use Automation System · ~14 minutes · every shot: where you are, what you type, what to say

Two windows only, and never anything else on screen: a **terminal** and **Chrome**. Each shot is tagged with which one you should be recording. The design reasoning behind all of this is in [docs/demo-narration.pdf](#) — this file is purely operational.

Timeline

#	Shot	Where	Length	The one point
0	Setup (not recorded)	Terminal	—	Reset state, start services
1	The legacy app	Chrome	1:00	This is what you have to automate
2	Discovery — learn G1	Terminal	1:30	The model is a compiler, not a runtime
3	The artifact	Terminal	0:45	A contract, not a transcript
4	Refusal → approve → replay	Both	1:30	2 seconds, no model; approval is a human decision
5	Three kinds of wrong	Terminal	2:00	Exit codes a caller can branch on
6	Human takes the live session	Chrome	3:00	Same session, resumed mid-flow
7	Learn something new, live	Chrome	2:00	It generalises — 25s to learn
8	The flow that commits	Both	2:30	One policy line = account opened or not
9	Safety and evidence	Terminal	1:00	Guardrails, and the honest limits
10	Close	—	0:30	The thesis in one sentence

0 · Setup

TERMINAL

NOT RECORDED

```
cd ~/orca/projects/portal-all-hands

# 1. free the ports — a stale process makes the engine die silently
lsof -ti:4100,4200,4300 | xargs kill -9 2>/dev/null

# 2. the live-learn goal must NOT exist yet (shot 7 depends on this)
rm -f artifacts/member-branch-and-tenure@1.json

# 3. undo any approvals a rehearsal left behind, then check the state
git checkout -- artifacts/
grep -o '"status": "[a-z]*"' artifacts/*.json
#   expect: open-subaccount approved, savings-balance@1 and @2 draft

# 4. health check, then start everything
pnpm cua doctor          # expect 8 ok, 1 warning, 0 failures
pnpm demo                # wait for all three "ready" lines
```

△ Record the **terminal window only**, never the full screen — your `.env` holds a live API key. Close any editor showing it. Font large enough to read at 1080p (14–16pt).

1 · The legacy app

CHROME

1:00

ON SCREEN localhost:4100 → sign in demo / demo → type 10042 → Search. Land on the Member Detail page and pause there.

SAY *"This is the kind of core banking UI that still runs in production and that nobody is going to rewrite. Framesets, table layouts, no API, no test ids. The value I want — this savings balance — is in an unlabelled table cell. Every design decision after this follows from that constraint."*

Optional, five seconds: right-click → View Source, scroll once, close it. It makes the "the model never sees raw HTML" line in shot 2 land harder.

2 · Discovery — learn G1

TERMINAL

1:30

```
pnpm cua discover \  
  --goal "Look up member {memberId} and read the current balance of their Savings account." \  
  --url http://localhost:4100/ --capability-id member-savings-balance --param memberId=10042
```

SAY *"I give it a goal in plain English and a URL. It signs in, searches, and reads the balance — about eight model calls, seven cents."*

While it runs, say the two things that matter:

SAY *"The model never sees raw HTML. It sees an accessibility tree flattened across the frames, plus a screenshot with every interactive element numbered — which is roughly what a human operator perceives. And {memberId} is a parameter, while the password is a reference to an environment variable. The literal value is substituted at the browser after the model has decided, so a credential never reaches the model, the transcript, or the artifact."*

3 · The artifact

TERMINAL

0:45

```
pnpm cua artifact summary artifacts/member-savings-balance@1.json
```

ON SCREEN The step list with <secret TARGET_PASSWORD> and {memberId} visible as references.

SAY *"That is the output — and it is not a transcript. It is a typed, versioned contract: declared inputs and outputs, every step with its locator fallbacks, what each step expects to be true afterwards, and a catalog of the conditions this flow knows how to handle. Someone can review and approve this without reading a log. That is the whole point."*

4a — terminal. Lead with the refusal. It is the beat people remember.

```
pnpm cua replay artifacts/member-savings-balance@2.json --param memberId=10042
# → FAILURE ARTIFACT_NOT_APPROVED (0 steps, 3ms)
```

SAY "A model worked this out once. That does not make it allowed to run unattended. Notice it refused in three milliseconds without even opening a browser."

4b — Chrome. Console → **Capabilities** → member-savings-balance@2 → **Approve for unattended replay.**

ON SCREEN Before clicking approve, scroll the contract once: inputs, outputs, secrets, origins, and a step showing its *fallbacks*: label → css → visual.

SAY "This is the review surface. Approval is a human decision made once, here, rather than on every run."

4c — terminal.

```
pnpm cua replay artifacts/member-savings-balance@2.json --param memberId=10042
# → SUCCESS {"savingsBalance":{"amount":1234.56,"currency":"USD"}} ~2s

pnpm cua replay artifacts/member-savings-balance@2.json --param memberId=10077
# → SUCCESS $8,900.04
```

SAY "Two seconds, no model, no API cost, and the same answer every time. The second member proves the parameter is real rather than baked into the recording."

Run all three, then talk. **Read the exit codes aloud** — that is the shot.

```
pnpm cua replay artifacts/member-savings-balance@2.json --param memberId=99999
#   → BUSINESS_OUTCOME MEMBER_NOT_FOUND          exit 0

pnpm cua replay artifacts/member-savings-balance@2.json --param memberId=10042 --fault session_e:
#   → SUCCESS, recovered via the login prelude      exit 0

pnpm cua replay artifacts/member-savings-balance@2.json --param memberId=10042 --fault session_e:
#   → FAILURE RECOVERY_LOOP                        exit 2
```

SAY *"Three different kinds of wrong. 'No such member' is an answer, not an error — the automation worked perfectly, so exit zero and the caller gets a typed business outcome. If that paged somebody, people would start ignoring the alerts. A session expiry is recoverable, so it re-runs the login and carries on, and the caller never learns it happened. The same expiry that never clears is bounded after two attempts and reported as a failure. A caller can branch on these without parsing a log."*

ON SCREEN Then: `cat evidence/replay-g1-recovery-loop/failure-*.md` — expected state, observed state, suggested next action.

SAY *"Every failure carries a bundle like this, with screenshots and an accessibility snapshot beside it."*

The most important shot in the video. Do not rush it.

6a — set it up. Console → **Start**:

Field	Value
Capability	member-savings-balance@2
Pace	Watchable — a beat between steps
Break something	Undeclared error on the next click
memberId	10042

△ Pace *must* be Watchable. At full speed the run finishes before the live viewport draws a single frame, and the live view is the most convincing thing in the demo.

Press **Replay**.

6b — it stops by itself. Wait. Do not touch anything.

ON SCREEN Banner: UNKNOWN_DIALOG: alert "Core banking link unavailable (ERR-7731)" · state **paused** · viewport labelled "**nobody is driving**" · a **Waiting** badge appears in the nav.

SAY "It stopped on its own. That dialog is not in this capability's outcome catalog, so it refuses to touch it. An automation that dismisses dialogs it does not recognise is an automation that will eventually dismiss something expensive."

The fault fires during sign-in, not at the search — the dialog attaches to the first click or focus. That is normal; don't be thrown by it.

6c — take over. Click **Take control**.

ON SCREEN State → **human control**; badge → "**you are driving**"; Accept / Dismiss buttons appear over the live frame.

SAY "Now I am driving the automation's own browser. Not a screenshot, not a copy — the same session, same cookies, same frames."

Click **Dismiss**. The badge turns to **LIVE** and you see the sign-in form.

6d — hand back. Click **Hand back** (mode: *Retry the step*). Wait for it to finish.

ON SCREEN Header: **completed** · **success**. Replay panel: **7 steps**, balance \$1,234.56 .

SAY "Seven steps. An uninterrupted run of this capability is also seven steps — so it resumed the step it stopped at, inside the session it paused in. It did not restart the browser, and the page it carried on from is the one my own clicks left behind."

6e — click Waiting in the nav.

ON SCREEN resolved · replay failure · same · by operator-1

SAY *"And the handover is in the audit trail: who claimed it, what they decided, when."*

SAY "Everything so far used capabilities I built beforehand. So let me teach it something it has never seen — and do it the way an operator would, from the console."

7a — Console → **Start** → **"Discover with a model"**. In the Goal dropdown choose **"Something else — describe it yourself"**, then fill in:

Field	Value
What should it do?	Look up member {memberId} and report which branch they belong to and the date they have been a member since.
Call it	member-branch-and-tenure
Model	Claude (uses your API key)
memberId	10042

ON SCREEN As you finish typing the goal, a **memberId** parameter box appears by itself, and the button's hint changes from "Needs a description, a name" to "Needs memberId".

SAY "Notice the parameter appeared on its own. Anything I put in braces becomes an input on the recorded capability — that is what makes it reusable for any member rather than a one-off for this one."

7b — press Discover and say nothing for a few seconds. Watch the right-hand panel, **Artifact taking shape**.

ON SCREEN The live viewport signs in and searches; the panel fills in step by step, and an **Outputs** entry appears: branch = "Main Street" via cellRightOfLabel.

SAY "That panel is the recipe being written as it works. Both values come from a different table than the balance did, and it has worked out a re-readable anchor for each — 'the cell to the right of the label Branch'. That anchor is what makes the replay deterministic."

Takes ~25 seconds, 8 model calls, about \$0.07. The run ends *completed* and the Evidence panel lists `artifact.json` alongside the screenshots.

7c — terminal. Prove the thing it just learned actually runs.

```
pnpm cua replay artifacts/member-branch-and-tenure@1.json --param memberId=10042 --allow-draft
#   → {"branch":"Main Street","memberSince":"2014-03-11"}    ~2.5s

pnpm cua replay artifacts/member-branch-and-tenure@1.json --param memberId=10077 --allow-draft
#   → {"branch":"Riverside","memberSince":"2018-09-30"}
```

SAY "Twenty-five seconds to learn it, two and a half to run it, and it generalises to a member it was never shown. That is the whole thesis: the expensive, non-deterministic part happens once."

SAY *"Everything so far only read data. This one opens an account."*

8a — terminal. Show the evidence of the real run rather than running it live (a live run here is 40 seconds of dead air).

```
cat evidence/discovery-g2-open-subaccount/run.json
python3 -m json.tool evidence/discovery-g2-open-subaccount/interventions.json | grep -E '"reason'
```

ON SCREEN 15 steps, 15 calls, \$0.14 — and **three** interventions.

SAY *"It stopped three times. Two were the policy: the button whose label matches the irreversible list, and the browser's own confirm dialog — which is the step that actually opens the account, so it is treated as a separate decision. The first one the model raised by itself, before any rule fired: it reached the review screen and asked for a human on its own initiative."*

8b — Chrome. Capabilities → `member-open-subaccount@1`, scroll to steps 7 and 8.

ON SCREEN Both tagged **risky · point of no return**, with the model's recorded reasoning underneath.

8c — terminal. The closing pair. Same artifact, same parameters.

```
pnpm cua replay artifacts/member-open-subaccount@1.json --param memberId=10042
# → SUCCESS {"confirmationNumber":"CU-7000xx"} side effects: committed exit 0

pnpm cua replay artifacts/member-open-subaccount@1.json --param memberId=10042 --risky block
# → FAILURE POLICY_VIOLATION side effects: none exit 2
```

SAY *"Same capability, same inputs, one line of policy different — and the difference is whether an account gets opened at all. The guardrail belongs to the deployment, not to the recorded flow."*

ON SCREEN Optional: refresh the member page in the bank UI to show the new sub-account.

SAY *"And the confirmation number changes on every run, because every run really does open an account. That is what 'committed' means here."*

```
pnpm cua doctor
```

SAY *"The secret check asks git, not .gitignore. Those two disagree exactly when a file was staged before the ignore rule was written — which is the case that actually leaks a key — and deleting it later does not help, because the history keeps it."*

```
cat policy.yaml      # scroll slowly  
ls evidence/
```

SAY *"Everything the engine may do is this one validated file, loaded by every entry point. Four layers enforce it: the model's decision, the surface boundary, the network, and a pre-flight check on the artifact. A human who takes control is not blocked by it — they are recorded as an override, because refusing them would disable escalation exactly when it is needed."*

Finish on the limit, out loud. It is a strength, not a weakness.

SAY *"And the honest part: screenshots are not masked. They render whatever was on screen, member IDs included. The structured evidence is redacted and there is a test that walks every file a run writes — but a grep cannot see inside a PNG, so the schema pins that flag to false and it is written down in the README. Everything here is synthetic data in an app I wrote."*

SAY *"So: an LLM works the task out once, under policy, with a human for anything irreversible. What it learned becomes a typed artifact that a person approves — and that artifact runs in two seconds with no model and no API cost. When the application does something it has not seen, it stops and hands a person the same live browser rather than guessing. The model is a compiler, not a runtime."*

If you want one more sentence, make it the honest one — it lands better than a feature:

SAY *"The most useful thing I found building this was that the flow that commits money was impossible for the system to complete, while the entire test suite was green. It only showed up when I drove the real application to its point of no return."*

If something goes wrong on camera

Symptom	Cause and fix
Engine exits immediately on <code>pnpm demo</code>	Stale process on 4200. <code>lsof -ti:4100,4200,4300 xargs kill -9</code> , then restart.
Shot 4 does not refuse	<code>member-savings-balance@2</code> was approved in a rehearsal — approving rewrites the file on disk. <code>git checkout -- artifacts/</code> puts it back. (A test asserts this, so a committed approval fails the suite rather than surprising you here.)
Shot 6 finishes before you can click	Pace was left at Full speed. Restart with <i>Watchable</i> .
Shot 7 says the capability already exists	You rehearsed it. <code>rm -f artifacts/member-branch-and-tenure@1.json</code> and delete its <code>evidence/run-*</code> directory.
Confirmation number is not CU-700001	Expected — it climbs within a session. Restarting the target app resets it. Say so; it proves the run committed.
Console shows "Connecting to the engine..."	The engine is not up yet. Wait for its ready line before switching windows.
A command runs stale code	You edited source since the last build. <code>pnpm --filter @cua/engine build</code> .

Cut-down versions

- **8 minutes** — drop shot 3 (fold the artifact into shot 4) and shot 9; shorten shot 5 to the first two commands.
- **5 minutes** — shots 1, 4, 6, 8 only. That is still the full argument: legacy UI, deterministic replay, human handover, guarded commit.
- **If you can only keep two** — shot 6 and shot 8.